

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЁТ
по индивидуальной практической работе №3
по дисциплине
«Интернет-технологии и веб-программирование»
Вариант 15

Выполнил

ст. гр. 220604
А.И. Михнюк

Проверила

Н.В. Хаджинова

Минск 2025

1 ЦЕЛЬ РАБОТЫ

Целью данной практической работы является освоение принципов модульного тестирования серверных приложений. В процессе выполнения работы происходит формирование понимания основных концепций и преимуществ модульного тестирования как методологии обеспечения качества программного обеспечения. Студент получает практические навыки установки и настройки фреймворка для тестирования *PHPUnit*, что является важным этапом в подготовке к профессиональной разработке.

Освоение процесса написания *unit*-тестов для отдельных компонентов *PHP*-приложения представляет собой значимую часть работы, поскольку позволяет на практике применить теоретические знания о тестировании функций и классов. Особое внимание уделяется использованию основных утверждений *PHPUnit* для проверки ожидаемого поведения кода, что составляет основу практического тестирования. Важным аспектом работы является развитие способности анализировать результаты выполнения тестов и находить ошибки в коде, что способствует формированию системного подхода к разработке программного обеспечения.

Приобретение навыков тестирования кода, взаимодействующего с базой данных, с использованием техник мокирования представляет собой продвинутый уровень подготовки, расширяющий профессиональные компетенции в области автоматизированного тестирования. Выполнение работы способствует пониманию роли тестирования в жизненном цикле разработки программного обеспечения и формированию культуры написания тестируемого кода. Практическое освоение этих аспектов подготавливает к решению реальных задач в профессиональной деятельности, где качество кода и его надежность являются критически важными параметрами.

2 ХОД РАБОТЫ

2.1 Подготовка и настройка окружения

Выполнение практической работы начинается с изучения теоретических основ модульного тестирования и ознакомления с принципами работы фреймворка *PHPUnit*. Изначально происходит установка необходимого программного обеспечения, включая систему контроля версий, интерпретатор *PHP* и менеджер зависимостей *Composer*. Установка *PHPUnit* осуществляется как зависимость проекта через *Composer*, что соответствует современным практикам разработки и позволяет управлять версиями тестирующих инструментов. Создание базовой конфигурации *PHPUnit* в файле *phpunit.xml* представляет собой важный этап настройки окружения для тестирования, где определяются пути к исходному коду и тестам, а также параметры выполнения тестов.

Разработка структуры проекта организуется согласно общепринятым стандартам, где исходный код размещается в директории *src*, а тесты — в директории *tests*. Такое разделение способствует поддержанию порядка в проекте и упрощает навигацию по кодовой базе. Структура папок внутри *tests* зеркально отражает структуру *src*, что обеспечивает логическую связь между тестами и тестируемым кодом. Именование файлов с тестами с суффиксом *Test* и наследование классов тестов от *PHPUnit\Framework\TestCase* являются стандартными требованиями фреймворка и соблюдение этих правил обеспечивает корректную работу тестовой инфраструктуры.

```
lenovo@DESKTOP-LK6T4GU MINGW64 /e/BSUIR/ITaWP/DistanceEducation2semester/ITaWP-Individual-practical-work-3 (master)
$ ./deploy.sh deploy
=== Начало развертывания приложения ===
Время: Sat Dec 6 22:39:46 2025

[INFO] Начало процесса развертывания...
[INFO] Docker и Docker Compose проверены успешно
[INFO] Остановка старых контейнеров...
[INFO] Сборка Docker образа...
time="2025-12-06T22:39:46+03:00" level=warning msg="E:\\BSUIR\\ITaWP\\DistanceEducation2semester\\ITaWP-Individual-practical-work-3\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
2025/12/06 22:39:46 http2: server: error reading preface from client //.pipe/dockerDesktopLinuxEngine: file has already been closed
[+] Building 95.3s (24/30)
=> [app internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 1.26kB 0.0s
=> [php-cli internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 1.26kB 0.0s
=> [php-cli internal] load metadata for docker.io/library/composer:latest 0.9s
=> [php-cli internal] load metadata for docker.io/library/php:8.2-apache 0.0s
=> [app auth] library/composer:pull token for registry-1.docker.io 0.0s
=> [app internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [php-cli internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
```

Рисунок 1 – Исполнение команды `./deploy.sh deploy`

2.2 Разработка тестируемых классов

Для реализации задания варианта 15 создаются два основных класса:

CurrencyConverter для конвертации валют и *MathHelper* для математических операций. Класс *CurrencyConverter* инкапсулирует логику работы с валютными курсами, предоставляя статичный набор обменных курсов и метод *convert* для выполнения конвертации. Важной особенностью реализации является обработка исключительных ситуаций, таких как передача неподдерживаемых кодов валют или попытка использования нулевого курса. Метод *addCurrency* позволяет расширять набор поддерживаемых валют, что добавляет гибкости классу. Класс *MathHelper* реализует базовые математические операции, включая сложение, вычитание, умножение и деление, а также дополнительные функции проверки четности чисел и вычисления факториала. Особое внимание уделяется обработке ошибок, в частности, генерации исключения при делении на ноль.

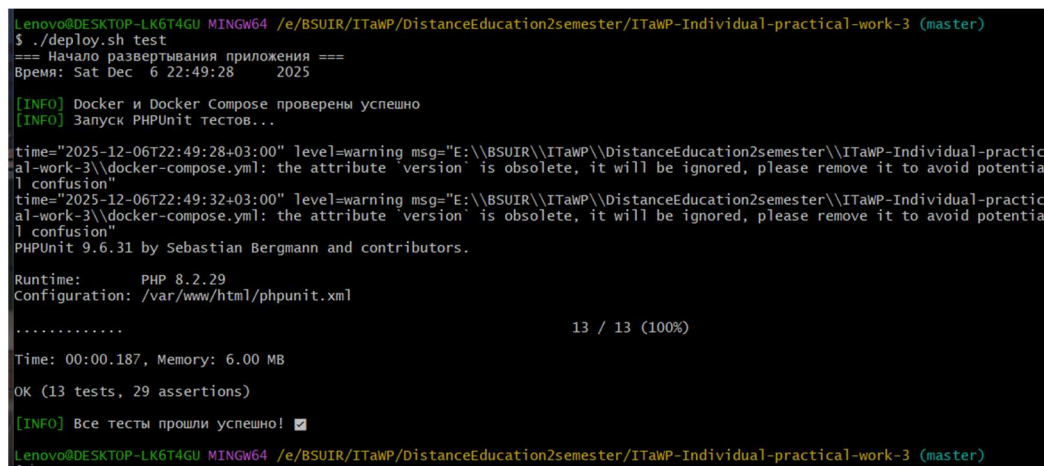
2.3 Создание тестовых сценариев

Написание тестов для этих классов представляет собой систематический процесс, начинающийся с создания тест-кейсов *CurrencyConverterTest* и *MathHelperTest*. Каждый тест-кейс содержит набор методов, покрывающих основную функциональность тестируемых классов, а также граничные случаи и ошибочные сценарии. Для класса *CurrencyConverter* разрабатываются тесты, проверяющие корректность конвертации между различными валютами, обработку ввода в разном регистре, конвертацию в ту же валюту, работу с нулевыми и отрицательными суммами. Отдельные тесты посвящены проверке реакции на неподдерживаемые валюты и добавление новых валютных пар. Тестирование класса *MathHelper* включает проверку базовых арифметических операций, обработку исключения при делении на ноль, проверку четности чисел и вычисление факториала с учетом граничных условий.

Использование методов-утверждений *PHPUnit* осуществляется в соответствии с их назначением: *assertEquals* проверяет равенство ожидаемого и фактического результатов, *assertTrue* и *assertFalse* используются для проверки булевых условий, *assertCount* контролирует количество элементов в массиве, *assertContains* проверяет наличие элемента в массиве. Для тестирования исключительных ситуаций применяется метод *expectException*, который гарантирует, что код генерирует ожидаемое исключение. Особое внимание уделяется использованию аннотации *@dataProvider* для параметризованных тестов, что позволяет тестировать один метод с различными наборами входных данных, увеличивая покрытие кода при уменьшении дублирования тестового кода.

Методы *setUp* и *tearDown* не нашли широкого применения в данном

проекте, поскольку тестируемые классы в основном используют статические методы, не требующие инициализации состояния между тестами. Однако в классе *CurrencyConverterTest* метод *setUp* используется для восстановления оригинальных курсов валют перед каждым тестом, что обеспечивает изоляцию тестов друг от друга и предотвращает влияние изменений состояния, сделанных в одном тесте, на результаты выполнения других тестов. Этот подход соответствует принципам независимости тестов и повышает их надежность.



```
Lenovo@DESKTOP-LK6T4GU MINGW64 /e/BSUIR/ITaWP/DistanceEducation2semester/ITaWP-Individual-practical-work-3 (master)
$ ./deploy.sh test
=== Начало развертывания приложения ===
Время: Sat Dec 6 22:49:28 2025

[INFO] Docker и Docker Compose проверены успешно
[INFO] Запуск PHPUnit тестов...

time="2025-12-06T22:49:28+03:00" level=warning msg="E:\\BSUIR\\ITaWP\\DistanceEducation2semester\\ITaWP-Individual-practical-work-3\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
time="2025-12-06T22:49:32+03:00" level=warning msg="E:\\BSUIR\\ITaWP\\DistanceEducation2semester\\ITaWP-Individual-practical-work-3\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
PHPUnit 9.6.31 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.29
Configuration: /var/www/html/phpunit.xml

.....                                     13 / 13 (100%)

Time: 00:00.187, Memory: 6.00 MB

OK (13 tests, 29 assertions)

[INFO] Все тесты прошли успешно! 🎉
Lenovo@DESKTOP-LK6T4GU MINGW64 /e/BSUIR/ITaWP/DistanceEducation2semester/ITaWP-Individual-practical-work-3 (master)
```

Рисунок 2 – Исполнение команды *./deploy.sh test*

2.4 Запуск тестов и анализ результатов

Запуск тестов осуществляется через командную строку с использованием скрипта *vendor/bin/phpunit*, что позволяет выполнять все тесты проекта или отдельные тест-кейсы. Анализ результатов тестирования включает интерпретацию вывода фреймворка, где зеленый цвет указывает на успешное прохождение тестов, а красный — на наличие ошибок или неудачных проверок. В случае обнаружения проблем проводится анализ причин их возникновения и внесение необходимых исправлений либо в тестируемый код, либо в сами тесты. Этот итеративный процесс способствует улучшению качества кода и тестов.

Тестирование с зависимостями в данной работе рассматривается на примере класса *CurrencyConverter*, который использует статичные данные о курсах валют. Хотя в данном случае не требуется мокирование внешних зависимостей, демонстрируется принцип изоляции тестируемого кода через использование фиктивных данных. Подготовка тестового окружения обеспечивает стабильность тестов независимо от изменений реальных

валютных курсов, что соответствует принципам модульного тестирования.

2.5 Автоматизация и контейнеризация

Для обеспечения воспроизводимости окружения и упрощения развертывания проекта разрабатывается контейнеризация приложения с использованием *Docker*. Создаются *Dockerfile* и *docker-compose.yml*, которые описывают конфигурацию контейнера с веб-сервером *Apache* и интерпретатором *PHP*. Контейнеризация позволяет развернуть приложение на любой системе, поддерживающей *Docker*, без необходимости ручной установки и настройки зависимостей. Дополнительно создается скрипт автоматического развертывания *deploy.sh*, который автоматизирует процесс сборки образа, запуска контейнеров, установки зависимостей и выполнения тестов. Этот скрипт адаптирован для различных операционных систем, включая *Windows 10*, где могут использоваться альтернативные способы запуска.

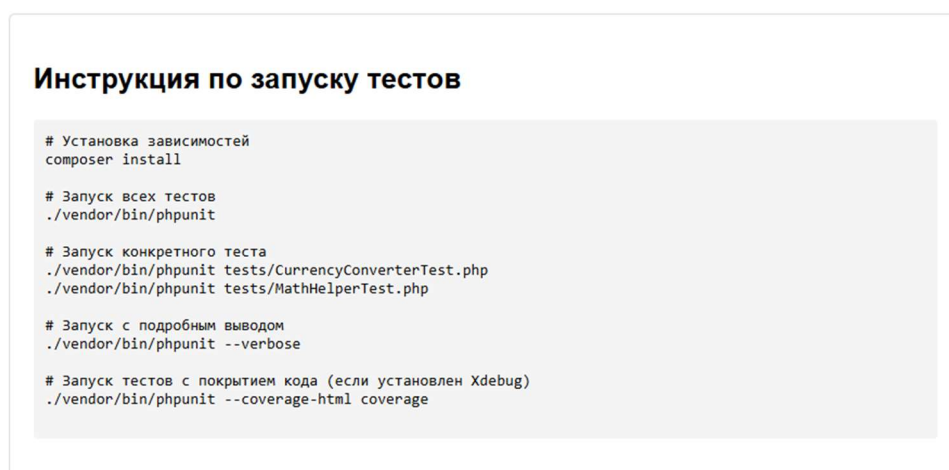


Рисунок 3 – Инструкция по запуску тестов

2.6 Запуск тестов и анализ результатов

Разработка веб-интерфейса в файле *public/index.php* служит демонстрационным целям, позволяя визуализировать работу тестируемых классов через браузер. Интерфейс отображает доступные валюты, примеры конвертации и результаты математических операций, предоставляя наглядное представление о функциональности приложения. Хотя основное внимание в работе уделяется тестированию, наличие веб-интерфейса показывает практическое применение разработанных классов в контексте веб-

приложения.

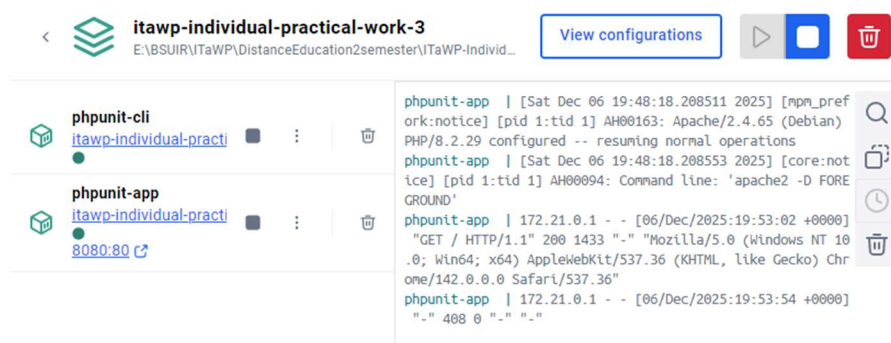


Рисунок 4 – Контейнеры в *Docker Desktop*

Документирование проекта осуществляется через файл *README.md*, содержащий описание структуры проекта, инструкции по установке и запуску, а также информацию о реализованных классах и тестах. Подробное документирование способствует пониманию проекта другими разработчиками и упрощает его сопровождение. Все этапы работы сопровождаются осмыслением выполняемых действий и их места в общем процессе разработки программного обеспечения, что способствует формированию целостного представления о роли тестирования в создании качественных программных продуктов.

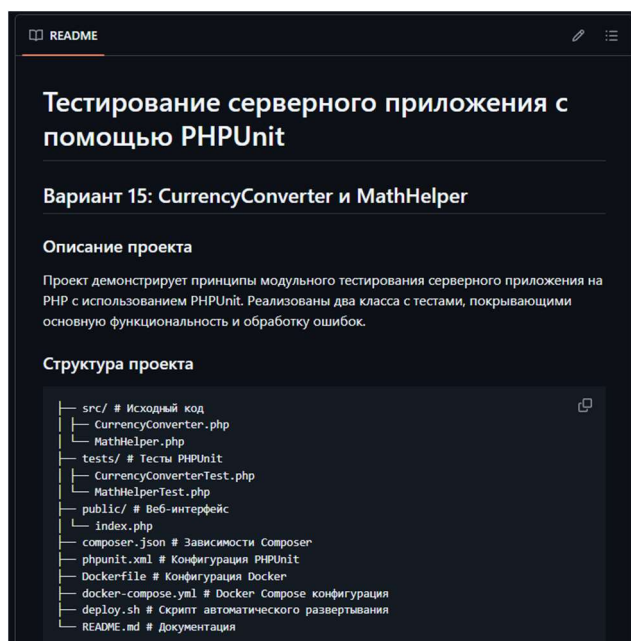


Рисунок 5 – Часть содержимого *README.md*

ЗАКЛЮЧЕНИЕ

Выполнение индивидуальной практической работы позволило достичь поставленных целей и получить комплексные практические навыки в области модульного тестирования серверных приложений с использованием фреймворка *PHPUnit*. В процессе работы были успешно разработаны и протестированы классы *CurrencyConverter* и *MathHelper*, покрытые набором тестов, проверяющих как основную функциональность, так и обработку исключительных ситуаций. Освоены принципы организации тестовой инфраструктуры, включая создание параметризованных тестов, использование методов-утверждений и обработку граничных случаев. Приобретенные компетенции включают умение анализировать результаты тестирования, находить ошибки в коде и применять подходы изоляции тестируемого кода от внешних зависимостей, что составляет основу профессионального подхода к разработке качественного программного обеспечения.

Практическая работа продемонстрировала важность автоматизации процессов разработки и тестирования через реализацию контейнеризации приложения с использованием *Docker* и создание скриптов автоматического развертывания. Контейнеризация обеспечила воспроизводимость окружения на различных операционных системах, включая *Windows 10*, что соответствует современным требованиям к разработке программного обеспечения. Разработанный веб-интерфейс визуализирует работу тестируемых классов, показывая их практическое применение в контексте веб-приложения. Автоматизация тестирования и развертывания способствует формированию культуры *DevOps*, где непрерывная интеграция и доставка становятся неотъемлемой частью жизненного цикла разработки.

Полученные знания и навыки создают прочную основу для дальнейшего изучения более продвинутых техник тестирования и методологий разработки. Практический опыт написания тестируемого кода, организации тестовой структуры проекта и анализа результатов тестирования будет полезен в последующей учебной и профессиональной деятельности. Работа способствует пониманию роли модульного тестирования в обеспечении качества программного обеспечения и формирует подход к разработке, ориентированный на создание надежных и поддерживаемых программных продуктов.

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код приложения

С полным кодом веб-приложения можно ознакомиться по ссылке:
<https://github.com/POOSTOTEL/ITaWP-Individual-practical-work-3>

Файл *insdex.php*

```
<?php
require_once __DIR__ . '/../src/CurrencyConverter.php';
require_once __DIR__ . '/../src/MathHelper.php';
?>
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Тестирование PHP приложения</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; }
        .container { max-width: 800px; margin: 0 auto; }
        .section { margin-bottom: 30px; padding: 20px; border: 1px solid #ddd; border-radius: 5px; }
    }
    pre { background: #f4f4f4; padding: 10px; border-radius: 3px; }
    .success { color: green; }
    .error { color: red; }
</style>
</head>
<body>
    <div class="container">
        <h1>Тестирование серверного приложения</h1>
        <p>Вариант 15: CurrencyConverter и MathHelper</p>
        <div class="section">
            <h2>Демонстрация работы CurrencyConverter</h2>
            <?php
            try {
                echo "<h3>Доступные валюты:</h3>";
                echo "<pre>";
                print_r(CurrencyConverter::getAvailableCurrencies());
                echo "</pre>";
                echo "<h3>Примеры конвертации:</h3>";
                $examples = [
                    ['amount' => 100, 'from' => 'USD', 'to' => 'EUR'],
```

```

        ['amount' => 50, 'from' => 'EUR', 'to' => 'GBP'],
        ['amount' => 1000, 'from' => 'GBP', 'to' => 'JPY'],
    ];
    foreach ($examples as $example) {
        $result = CurrencyConverter::convert(
            $example['amount'],
            $example['from'],
            $example['to']
        );
        echo "<p>{$example['amount']} {$example['from']} = "
            . number_format($result, 2) . " {$example['to']}</p>";
    }
    echo "<p class='success'>Конвертация выполнена успешно!</p>";
} catch (Exception $e) {
    echo "<p class='error'>Ошибка: " . $e->getMessage() . "</p>";
}
?>
</div>
<div class="section">
    <h2>Демонстрация работы MathHelper</h2>
    <?php
        echo "<h3>Математические операции:</h3>";
        echo "<p>Сложение: 2 + 3 = " . MathHelper::add(2, 3) . "</p>";
        echo "<p>Вычитание: 5 - 2 = " . MathHelper::subtract(5, 2) . "</p>";
        echo "<p>Умножение: 4 * 3 = " . MathHelper::multiply(4, 3) . "</p>";
        echo "<p>Деление: 10 / 2 = " . MathHelper::divide(10, 2) . "</p>";
        echo "<h3>Дополнительные функции:</h3>";
        echo "<p>Четность числа 4: " . (MathHelper::isEven(4) ? 'четное' : 'нечетное') .
            "</p>";
        echo "<p>Четность числа 7: " . (MathHelper::isEven(7) ? 'четное' : 'нечетное') .
            "</p>";
        echo "<p>Факториал 5: " . MathHelper::factorial(5) . "</p>";
    ?>
</div>
<div class="section">
    <h2>Инструкция по запуску тестов</h2>
    <pre>
# Установка зависимостей
composer install
# Запуск всех тестов
./vendor/bin/phpunit
# Запуск конкретного теста
./vendor/bin/phpunit tests/CurrencyConverterTest.php
./vendor/bin/phpunit tests/MathHelperTest.php
# Запуск с подробным выводом
./vendor/bin/phpunit --verbose

```

```
# Запуск тестов с покрытием кода (если установлен Xdebug)
./vendor/bin/phpunit --coverage-html coverage
</pre>
</div>
</div>
</body>
</html>
```

Файл *CurrencyConverter.php*

```
<?php
class CurrencyConverter
{
    private static $exchangeRates = [
        'USD' => 1.0,
        'EUR' => 0.85,
        'GBP' => 0.75,
        'JPY' => 110.0,
        'RUB' => 75.0
    ];
    /**
     * Конвертирует сумму из одной валюты в другую
     *
     * @param float $amount Сумма для конвертации
     * @param string $fromCurrency Исходная валюта (код из 3 букв)
     * @param string $toCurrency Целевая валюта (код из 3 букв)
     * @return float Конвертированная сумма
     * @throws InvalidArgumentException Если валюта не поддерживается
     * @throws DivisionByZeroError Если курс валюты равен 0
     */
    public static function convert(float $amount, string $fromCurrency, string $toCurrency): float
    {
        $fromCurrency = strtoupper($fromCurrency);
        $toCurrency = strtoupper($toCurrency);
        if (!isset(self::$exchangeRates[$fromCurrency])) {
            throw new InvalidArgumentException("Неподдерживаемая исходная валюта: {$fromCurrency}");
        }
        if (!isset(self::$exchangeRates[$toCurrency])) {
            throw new InvalidArgumentException("Неподдерживаемая целевая валюта: {$toCurrency}");
        }
        $rateFrom = self::$exchangeRates[$fromCurrency];
        $rateTo = self::$exchangeRates[$toCurrency];
        if ($rateFrom == 0) {
```

```

        throw new DivisionByZeroError("Курс исходной валюты не может быть равен 0");
    }
    $amountInUsd = $amount / $rateFrom;
    return $amountInUsd * $rateTo;
}
/**
 * Добавляет новую валюту или обновляет существующий курс
 *
 * @param string $currency Код валюты
 * @param float $rate Курс к USD
 * @throws InvalidArgumentException Если курс отрицательный
 */
public static function addCurrency(string $currency, float $rate): void
{
    $currency = strtoupper($currency);

    if ($rate < 0) {
        throw new InvalidArgumentException("Курс валюты не может быть отрицательным");
    }
    self::$exchangeRates[$currency] = $rate;
}
/**
 * Возвращает все доступные курсы валют
 *
 * @return array Массив валют и их курсов
 */
public static function getAvailableCurrencies(): array
{
    return self::$exchangeRates;
}
}

```

Файл *MathHelper.php*

```

<?php
class MathHelper
{
    /**
     * Сложение двух чисел
     *
     * @param float $a Первое число
     * @param float $b Второе число
     * @return float Результат сложения
     */

```

```

public static function add(float $a, float $b): float
{
    return $a + $b;
}
/**
 * Вычитание двух чисел
 *
 * @param float $a Первое число
 * @param float $b Второе число
 * @return float Результат вычитания
 */
public static function subtract(float $a, float $b): float
{
    return $a - $b;
}
/**
 * Умножение двух чисел
 *
 * @param float $a Первое число
 * @param float $b Второе число
 * @return float Результат умножения
 */
public static function multiply(float $a, float $b): float
{
    return $a * $b;
}
/**
 * Деление двух чисел
 *
 * @param float $a Делимое
 * @param float $b Делитель
 * @return float Результат деления
 * @throws DivisionByZeroError Если делитель равен 0
 */
public static function divide(float $a, float $b): float
{
    if ($b == 0) {
        throw new DivisionByZeroError("Деление на ноль невозможно");
    }

    return $a / $b;
}
/**
 * Проверка, является ли число четным
 *
 * @param int $number Число для проверки

```

```

    * @return bool true если четное, false если нечетное
    */
    public static function isEven(int $number): bool
    {
        return $number % 2 == 0;
    }
    /**
     * Вычисление факториала числа
     *
     * @param int $n Число
     * @return int Факториал числа
     * @throws InvalidArgumentException Если число отрицательное
     */
    public static function factorial(int $n): int
    {
        if ($n < 0) {
            throw new InvalidArgumentException("Факториал отрицательного числа не
определен");
        }
        $result = 1;
        for ($i = 2; $i <= $n; $i++) {
            $result *= $i;
        }
        return $result;
    }
}

```

Файл CurrencyConverter.php

```

<?php
use PHPUnit\Framework\TestCase;
require_once __DIR__ . '/../src/CurrencyConverter.php';
class CurrencyConverterTest extends TestCase
{
    protected function setUp(): void
    {
        CurrencyConverter::addCurrency('USD', 1.0);
        CurrencyConverter::addCurrency('EUR', 0.85);
        CurrencyConverter::addCurrency('GBP', 0.75);
        CurrencyConverter::addCurrency('JPY', 110.0);
        CurrencyConverter::addCurrency('RUB', 75.0);
    }
    public function testConvertUsdToEur(): void
    {
        $result = CurrencyConverter::convert(100, 'USD', 'EUR');
    }
}

```

```

    $this->assertEquals(85.0, $result);
}
public function testConvertEurToUsd(): void
{
    $result = CurrencyConverter::convert(100, 'EUR', 'USD');
    $this->assertEqualsWithDelta(117.647, $result, 0.001);
}
public function testConvertWithDifferentCase(): void
{
    $result = CurrencyConverter::convert(100, 'usd', 'eur');
    $this->assertEquals(85.0, $result);
}
public function testConvertSameCurrency(): void
{
    $result = CurrencyConverter::convert(100, 'USD', 'USD');
    $this->assertEquals(100, $result);
}
public function testConvertZeroAmount(): void
{
    $result = CurrencyConverter::convert(0, 'USD', 'EUR');
    $this->assertEquals(0, $result);
}
public function testConvertNegativeAmount(): void
{
    $result = CurrencyConverter::convert(-100, 'USD', 'EUR');
    $this->assertEquals(-85.0, $result);
}
public function testInvalidFromCurrency(): void
{
    $this->expectException(InvalidArgumentException::class);
    $this->expectExceptionMessage('Не поддерживаемая исходная валюта: XYZ');

    CurrencyConverter::convert(100, 'XYZ', 'USD');
}
public function testInvalidToCurrency(): void
{
    $this->expectException(InvalidArgumentException::class);
    $this->expectExceptionMessage('Не поддерживаемая целевая валюта: XYZ');

    CurrencyConverter::convert(100, 'USD', 'XYZ');
}
public function testAddNewCurrency(): void
{
    CurrencyConverter::addCurrency('CNY', 6.5);
    $result = CurrencyConverter::convert(100, 'USD', 'CNY');
    $this->assertEquals(650.0, $result);
}

```

```

}
public function testAddCurrencyWithNegativeRate(): void
{
    $this->expectException(InvalidArgumentException::class);
    $this->expectExceptionMessage('Курс валюты не может быть отрицательным');
    CurrencyConverter::addCurrency('TEST', -1.0);
}
public function testAddCurrencyWithZeroRate(): void
{
    CurrencyConverter::addCurrency('ZERO', 0.0);
    $this->expectException(DivisionByZeroError::class);
    CurrencyConverter::convert(100, 'ZERO', 'USD');
}
public function testGetAvailableCurrencies(): void
{
    $currencies = CurrencyConverter::getAvailableCurrencies();
    $this->assertIsArray($currencies);
    $this->assertArrayHasKey('USD', $currencies);
    $this->assertArrayHasKey('EUR', $currencies);
    $this->assertCount(5, $currencies);
}
public function testConvertComplexChain(): void
{
    $result1 = CurrencyConverter::convert(100, 'USD', 'EUR');
    $result2 = CurrencyConverter::convert($result1, 'EUR', 'GBP');
    $result3 = CurrencyConverter::convert($result2, 'GBP', 'USD');
    $this->assertEqualsWithDelta(100, $result3, 0.01);
}
/**
 * @dataProvider conversionProvider
 */
public function testMultipleConversions(float $amount, string $from, string $to, float
$expected): void
{
    $result = CurrencyConverter::convert($amount, $from, $to);
    $this->assertEqualsWithDelta($expected, $result, 0.01);
}
public function conversionProvider(): array
{
    return [
        [100, 'USD', 'EUR', 85.0],
        [50, 'EUR', 'GBP', 44.12],
        [1000, 'GBP', 'JPY', 146666.67],
        [5000, 'JPY', 'RUB', 3409.09],
    ];
}

```



```

    public function testIncompleteTest(): void
    {
        $this->markTestIncomplete('Требуется реализация конвертации с плавающим курсом');
    }
    public function testSkippedTest(): void
    {
        $this->markTestSkipped('Пропускаем из-за проблем с API');
    }
}

```

Файл *MathHelperTest.php*

```

<?php
use PHPUnit\Framework\TestCase;
require_once __DIR__ . '/../src/MathHelper.php';
class MathHelperTest extends TestCase
{
    public function testAdd(): void
    {
        $this->assertEquals(5, MathHelper::add(2, 3));
        $this->assertEquals(0, MathHelper::add(-2, 2));
        $this->assertEquals(1.5, MathHelper::add(1, 0.5));
    }
    public function testSubtract(): void
    {
        $this->assertEquals(1, MathHelper::subtract(3, 2));
        $this->assertEquals(-1, MathHelper::subtract(2, 3));
        $this->assertEquals(0.5, MathHelper::subtract(1, 0.5));
    }
    public function testMultiply(): void
    {
        $this->assertEquals(6, MathHelper::multiply(2, 3));
        $this->assertEquals(0, MathHelper::multiply(2, 0));
        $this->assertEquals(-6, MathHelper::multiply(2, -3));
    }
    public function testDivide(): void
    {
        $this->assertEquals(2, MathHelper::divide(6, 3));
        $this->assertEquals(0.5, MathHelper::divide(1, 2));
        $this->assertEquals(-2, MathHelper::divide(6, -3));
    }
    public function testDivideByZero(): void
    {
        $this->expectException(DivisionByZeroError::class);
    }
}

```

```

    $this->expectExceptionMessage('Деление на ноль невозможно');

    MathHelper::divide(5, 0);
}
public function testIsEven(): void
{
    $this->assertTrue(MathHelper::isEven(2));
    $this->assertTrue(MathHelper::isEven(0));
    $this->assertFalse(MathHelper::isEven(3));
    $this->assertFalse(MathHelper::isEven(-1));
}
public function testFactorial(): void
{
    $this->assertEquals(1, MathHelper::factorial(0));
    $this->assertEquals(1, MathHelper::factorial(1));
    $this->assertEquals(120, MathHelper::factorial(5));
}
public function testFactorialNegative(): void
{
    $this->expectException(InvalidArgumentException::class);
    $this->expectExceptionMessage('Факториал отрицательного числа не определен');

    MathHelper::factorial(-5);
}
/**
 * @dataProvider additionProvider
 */
public function testAddWithDataProvider(float $a, float $b, float $expected): void
{
    $this->assertEquals($expected, MathHelper::add($a, $b));
}
public function additionProvider(): array
{
    return [
        [1, 1, 2],
        [0, 0, 0],
        [-1, 1, 0],
        [2.5, 2.5, 5.0],
    ];
}
public function testAssertContains(): void
{
    $numbers = [1, 2, 3, 5, 8];
    $this->assertContains(5, $numbers);
    $this->assertNotContains(4, $numbers);
}

```

```
}
```

Файл *composer.json*

```
{
  "name": "student/phpunit-project",
  "description": "PHPUnit тестирование для студенческого задания",
  "type": "project",
  "require": {
    "php": "^7.4 || ^8.0"
  },
  "require-dev": {
    "phpunit/phpunit": "^9.5"
  },
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  },
  "scripts": {
    "test": "phpunit",
    "test-coverage": "phpunit --coverage-html coverage"
  }
}
```

Файл *deploy.sh*

```
#!/bin/bash
set -e
echo "=== Начало разворачивания приложения ==="
echo "Время: $(date)"
echo ""
GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
NC='\033[0m' # No Color
print_message() {
  echo -e "${GREEN}[INFO]${NC} $1"
}
print_error() {
  echo -e "${RED}[ERROR]${NC} $1"
}
print_warning() {
  echo -e "${YELLOW}[WARNING]${NC} $1"
}
```

```

check_docker() {
    if ! command -v docker &> /dev/null; then
        print_error "Docker не установлен. Установите Docker перед продолжением."
        exit 1
    fi
    if ! command -v docker-compose &> /dev/null; then
        print_error "Docker Compose не установлен. Установите Docker Compose перед продолжением."
        exit 1
    fi
    print_message "Docker и Docker Compose проверены успешно"
}

build_image() {
    print_message "Сборка Docker образа..."
    docker-compose build --no-cache
    if [ $? -eq 0 ]; then
        print_message "Образ успешно собран"
    else
        print_error "Ошибка при сборке образа"
        exit 1
    fi
}

start_containers() {
    print_message "Запуск контейнеров..."
    docker-compose up -d
    sleep 5
    if [ $(docker-compose ps | grep -c "Up") -ge 1 ]; then
        print_message "Контейнеры успешно запущены"
    else
        print_error "Ошибка при запуске контейнеров"
        exit 1
    fi
}

install_dependencies() {
    print_message "Установка зависимостей Composer..."
    docker-compose exec app composer install --no-interaction --no-scripts
    if [ $? -eq 0 ]; then
        print_message "Зависимости успешно установлены"
    else
        print_error "Ошибка при установке зависимостей"
        exit 1
    fi
}

run_tests() {
    print_message "Запуск PHPUnit тестов..."
    echo ""
}

```

```

if docker-compose ps | grep -q "php-cli"; then
    docker-compose exec php-cli ./vendor/bin/phpunit --verbose
else
    docker-compose exec app ./vendor/bin/phpunit --verbose
fi
TEST_RESULT=$?
echo ""
if [ $TEST_RESULT -eq 0 ]; then
    print_message "Все тесты прошли успешно!"
else
    print_warning "Некоторые тесты не прошли. Проверьте вывод выше."
fi
}
check_health() {
    print_message "Проверка здоровья приложения..."
    sleep 3
    if curl -s -f http://localhost:8080 > /dev/null; then
        print_message "Веб-приложение доступно по адресу: http://localhost:8080"
    else
        print_warning "Веб-приложение недоступно"
    fi
}
show_deployment_info() {
    echo ""
    echo "===== "
    echo "РАЗВЕРТЫВАНИЕ ЗАВЕРШЕНО УСПЕШНО!"
    echo "===== "
    echo ""
    echo "Доступные точки входа:"
    echo "1. Веб-интерфейс: http://localhost:8080"
    echo "2. Запуск тестов вручную:"
    echo "  docker-compose exec app ./vendor/bin/phpunit"
    echo ""
    echo "Команды управления:"
    echo "• Запуск: docker-compose up -d"
    echo "• Остановка: docker-compose down"
    echo "• Просмотр логов: docker-compose logs -f"
    echo "• Пересборка: docker-compose build --no-cache"
    echo ""
    echo "Структура проекта:"
    echo "• src/ - Исходный код приложения"
    echo "• tests/ - Тесты PHPUnit"
    echo "• public/ - Веб-интерфейс"
    echo ""
    echo "Для выхода нажмите Ctrl+C"
}

```

```

    echo "=====
}
deploy() {
    print_message "Начало процесса развертывания..."
    check_docker
    print_message "Остановка старых контейнеров..."
    docker-compose down 2>/dev/null || true
    build_image
    start_containers
    install_dependencies
    run_tests
    check_health
    show_deployment_info
    log_deployment
}

log_deployment() {
    LOG_FILE="deployment.log"
    echo "=== Лог развертывания ===" >> $LOG_FILE
    echo "Дата: $(date)" >> $LOG_FILE
    echo "Пользователь: $(whoami)" >> $LOG_FILE
    echo "Система: $(uname -a)" >> $LOG_FILE
    echo "Docker версия: $(docker --version)" >> $LOG_FILE
    echo "Docker Compose версия: $(docker-compose --version)" >> $LOG_FILE
    echo "Статус: УСПЕШНО" >> $LOG_FILE
    echo "===== " >> $LOG_FILE
    print_message "Лог развертывания сохранен в $LOG_FILE"
}

case "$1" in
    "deploy")
        deploy
        ;;
    "test")
        check_docker
        run_tests
        ;;
    "build")
        check_docker
        build_image
        ;;
    "start")
        check_docker
        start_containers
        ;;
    "stop")
        docker-compose down

```

```

    print_message "Контейнеры остановлены"
    ;;
"logs")
    docker-compose logs -f
    ;;
"clean")
    docker-compose down -v
    docker system prune -f
    print_message "Система очищена"
    ;;
*)
    echo "Использование: $0 {deploy|test|build|start|stop|logs|clean}"
    echo ""
    echo "Команды:"
    echo " deploy - Полное развертывание приложения"
    echo " test   - Запуск тестов"
    echo " build  - Сборка Docker образа"
    echo " start  - Запуск контейнеров"
    echo " stop   - Остановка контейнеров"
    echo " logs   - Просмотр логов"
    echo " clean  - Очистка системы"
    exit 1
    ;;
Esac

```

Файл *docker-compose.yml*

```

version: '3.8'
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: phpunit-app
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./var/www/html
      - ./php.ini:/usr/local/etc/php/conf.d/custom.ini
    environment:
      - APACHE_RUN_USER=www-data
      - APACHE_RUN_GROUP=www-data
      - APACHE_LOG_DIR=/var/log/apache2
    networks:

```

```

    - app-network
php-cli:
build:
    context: .
    dockerfile: Dockerfile
container_name: phpunit-cli
command: tail -f /dev/null
volumes:
    - ./var/www/html
working_dir: /var/www/html
networks:
    - app-network
networks:
app-network:
    driver: bridge

```

Файл Dockerfile

```

FROM php:8.2-apache
RUN apt-get update && apt-get install -y \
    git \
    unzip \
    curl \
    libzip-dev \
    && docker-php-ext-install zip \
    && docker-php-ext-enable zip
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
RUN a2enmod rewrite
COPY . /var/www/html/
WORKDIR /var/www/html
RUN composer install --no-interaction --no-scripts
RUN chown -R www-data:www-data /var/www/html \
    && chmod -R 755 /var/www/html
RUN echo "ServerName localhost" >> /etc/apache2/apache2.conf
RUN sed -i 's!/var/www/html!/var/www/html/public!g' /etc/apache2/sites-available/000-
default.conf
EXPOSE 80
CMD ["apache2-foreground"]

```